F1 Deep Audit

Everything I Need to Know as an AI PM

Feature: F1 — Regulatory Feed Monitoring

Status: Complete | Audited: 2026-06-01

RegWatch AI Build Session

This is a permanent reference document for F1, written by reading every line of every file in the codebase — not from memory. Use it to understand what was built, every decision that was made, and how to explain it.

Section	Topic
1	Project File Map — every file, what it does, what breaks if deleted
2	Data Flow — one real document traced end to end through every function
3	Every AI/ML Decision — technique, input, output, failure modes
4	Every Architectural Decision — chosen path, alternative, risk level
5	What Is Good, Weak, and Missing — honest assessment
6	8 Interview-Ready PM Answers — with specific code references
7	Architecture Diagram — full ASCII system map
—	Summary Scorecard — Engineering, AI/ML, Production, PM Explainability

SECTION 1 — Project File Map

Every file in the F1 codebase. For each file: what it does, its most important function, what connects to it, what breaks if deleted, and why it was built this way.

FILE: src/models.py

DOES	Defines all three database tables as Python classes — Agency, RegulatoryDocument, AuditLog.
KEY	RegulatoryDocument — the atomic unit of the product. Every feature F1-F5 reads/writes here.
CONNECTS TO	Every file in the project imports from here. database.py creates tables. f1_ingest/*.py all import RegulatoryDocument.
BREAKS IF DELETED	Everything stops. No tables, no imports, no pipeline.
WHY THIS WAY	SQLModel merges SQLAlchemy (DB engine) and Pydantic (validation) into one class — eliminates drift risk between DB schema and Python schema.

FILE: src/database.py

DOES	Creates the DB engine, provides get_session() context manager, exposes create_db_and_tables().
KEY	get_session() — context manager that yields a DB session and guarantees cleanup even on exceptions.
CONNECTS TO	Imported by every file that touches the DB: agencies.py, dedup.py, anomaly.py, fulltext.py, ingest.py, health.py, query.py, dashboard/app.py.
BREAKS IF DELETED	All DB operations fail. No reads, no writes.
WHY THIS WAY	Separating connection management from models means models can be imported in tests without a live DB connection. DATABASE_URL from .env — swap SQLite to Postgres with one env var change.

FILE: src/f1_ingest/agencies.py

DOES	Defines the 6 agency feed configs as Python dicts and provides seed_agencies() to write them to DB.
KEY	seed_agencies() — idempotent insert: checks slug uniqueness before inserting, safe to run many times.
CONNECTS TO	scripts/setup_db.py calls seed_agencies(). ingest.py uses FR_API_SLUGS to route agencies to correct fetcher.
BREAKS IF DELETED	setup_db.py fails. Agency table empty. Ingest pipeline finds no agencies and exits immediately.
WHY THIS WAY	Storing in DB (not YAML) means a future UI can toggle agencies without a code deploy. active=False disables without deleting.

FILE: src/f1_ingest/fetcher.py

DOES	Two fetchers — fetch_feed() for RSS (Fed), fetch_fr_api() for FR JSON API (CFPB, OCC, FDIC, FinCEN).
KEY	fetch_fr_api() — handles 4 agencies whose RSS feeds block automated requests using the public FR JSON API.
CONNECTS TO	Called by ingest.py. Imports classifier.py, dedup.py, models.py.
BREAKS IF DELETED	Ingest pipeline cannot fetch any documents. 0 documents per run.
WHY THIS WAY	FR RSS feeds return HTML gate pages (200 but no XML). FR JSON API is public and stable. Two focused fetchers beats one complex adaptive fetcher.

FILE: src/f1_ingest/classifier.py

DOES	Keyword-matches a document title against ordered rule lists and returns a DocType enum value.
KEY	classify_doc_type(title: str) -> DocType — loops _RULES in priority order, first match wins, falls back to OTHER.
CONNECTS TO	Called inside fetch_feed() and fetch_fr_api() for every document parsed. Tested in test_f1_classifier.py.
BREAKS IF DELETED	Every document ingested as DocType.OTHER. Dashboard doc type filtering useless. F4 task priority breaks in Week 5.
WHY THIS WAY	Rule-based: zero cost, zero latency, explainable, no training data needed Day 1. F2 LLM reclassifies with full text.

FILE: src/f1_ingest/dedup.py

DOES	Computes SHA-256(title + url) as a content fingerprint and checks if that hash exists in the DB.
KEY	is_duplicate(content_hash: str) -> bool — single DB lookup, returns True/False.
CONNECTS TO	Called in ingest.py before every save. compute_hash() called inside fetcher.py when building each document.
BREAKS IF DELETED	Every document saved every run, including re-runs. 9 duplicates appear. F2 summarises same doc twice.
WHY THIS WAY	SHA-256(title+url) handles both failure modes: same doc at different URLs AND same URL cross-posted. Proved Day 2: 9 caught.

FILE: src/f1_ingest/fulltext.py

DOES	Fetches complete regulation text. FR API raw_text_url for Federal Register docs; BeautifulSoup for Fed HTML.
KEY	run_fulltext_enrichment(limit) — batch enriches docs with short raw_content, oldest first, 1 sec rate limit.
CONNECTS TO	Called in ingest.py after new docs saved. Called directly by scripts/enrich_fulltext.py.
BREAKS IF DELETED	raw_content stays as 1-2 sentence abstracts. F2 quality collapses. Near-duplicate detection lost.
WHY THIS WAY	Two strategies because two source types: FR has clean raw_text_url endpoint; Fed is HTML requiring BeautifulSoup parsing.

FILE: src/f1_ingest/anomaly.py

DOES	Runs Z-score volume + off-schedule day-of-week anomaly detection on newly ingested documents.
KEY	run_anomaly_check(new_docs) -> int — orchestrates both detectors, flags docs, writes AuditLog, returns count.
CONNECTS TO	Called in ingest.py after save and enrichment. Writes is_anomaly=True to RegulatoryDocument. Dashboard reads is_anomaly.
BREAKS IF DELETED	No anomaly detection. is_anomaly stays False. Sarah never gets proactive alerts about publication spikes.
WHY THIS WAY	Z-score over Isolation Forest: we have 1 day of history (IF needs weeks), Z-score is explainable, required for compliance.

FILE: src/f1_ingest/ingest.py

DOES	The pipeline orchestrator. Loads agencies, routes to fetcher, deduplicates, saves, enriches, anomaly checks, logs.
KEY	run_ingest(agency_slugs=None) -> dict — runs complete pipeline for all active agencies, returns summary dict.
CONNECTS TO	Imports and calls every other F1 component. Called by scripts/run_daily.py and daily_validate.py.
BREAKS IF DELETED	Pipeline cannot run. All components exist but nothing coordinates them.
WHY THIS WAY	Separation of orchestration from business logic: ingest.py has no logic of its own — it only coordinates. Each component independently testable.

FILE: src/f1_ingest/health.py

DOES	Two read-only checks per agency: reachability (feed reachable + >=1 entry?) and freshness (docs within 3 days?).
KEY	AgencyHealth dataclass with healthy, status_label, last_doc_date properties.
CONNECTS TO	Called by scripts/run_daily.py and daily_validate.py. Does not write to DB.
BREAKS IF DELETED	No feed monitoring. Silent failures go undetected. 0 docs ingested and nobody knows.
WHY THIS WAY	Two checks because a feed can return 200 with empty content (exactly what FR RSS feeds did). Freshness catches silent failures.

FILE: src/f1_ingest/query.py

DOES	CLI tool to inspect DB contents — summary stats by agency/doc type, recent docs, anomaly-flagged view.
KEY	show_summary() — prints total documents, agency/type breakdown, anomaly/review counts.
CONNECTS TO	Reads RegulatoryDocument from DB. No writes. Standalone — not called by any other file.
BREAKS IF DELETED	No CLI inspection tool. Must use DB Browser for SQLite to see data.
WHY THIS WAY	Alternative was waiting for Streamlit. Query tool gives immediate visibility during development.

FILE: dashboard/app.py

DOES	Streamlit browser dashboard. Loads all docs, applies agency/doc-type/anomaly filters, renders document cards.
KEY	load_documents() — @st.cache_data(ttl=300) cached DB query returning docs as plain dicts for Streamlit.
CONNECTS TO	Imports database.py, models.py, dashboard/components.py. Reads RegulatoryDocument. No writes.
BREAKS IF DELETED	No browser UI. Week 1 exit gate unmet — Mike cannot view filtered feeds.
WHY THIS WAY	Streamlit over React: React needs npm, build tooling, API layer, CORS — weeks of work. Streamlit is one Python file.

FILE: dashboard/components.py

DOES	Reusable UI helpers — colour-coded doc type badges, anomaly badge, KPI metric row, document card renderer.
KEY	render_document_card(doc) — expandable Streamlit card with badges, content preview, source link.
CONNECTS TO	Imported by dashboard/app.py. No other dependencies.
BREAKS IF DELETED	app.py cannot render anything — all display imports fail.
WHY THIS WAY	Separating display from data logic means React replacement in Week 6 only changes this file, not app.py.

FILE: fixtures/golden/f1_golden_set.json

DOES	10 hand-labeled regulatory documents with ground-truth fields. The acceptance test for F2.
KEY	eval_instructions field — defines faithfulness, relevance, and passing threshold (RAGAS >=0.85/0.80).
CONNECTS TO	Will be read by F2 eval harness in Week 3. Currently standalone.
BREAKS IF DELETED	F2 has no ground truth. Cannot objectively say F2 is done.
WHY THIS WAY	Hand-labeled from real document content, not LLM-generated. LLM labels would measure self-consistency, not accuracy.

FILE: docs/FP-FN-Risk-Matrix.md

DOES	Quantifies business cost of false negatives (missed regs = $500K-$5M fines) vs false positives (wasted time).
KEY	Asymmetry Summary — FN costs millions, FP costs 30 minutes. Drives every threshold decision.
CONNECTS TO	Referenced in design decisions for anomaly threshold, confidence threshold, HITL gate. Not imported by code.
BREAKS IF DELETED	Design decisions lose their written rationale. Future threshold changes made without understanding cost asymmetry.
WHY THIS WAY	Written before F2 is built — eval-first principle. Defining failure costs before building ensures correct thresholds.

SECTION 2 — Data Flow: One Document End to End

Document: **Federal Reserve Board issues enforcement actions with former employee of Atlantic Union Bank and former employee of Frost Bank** (May 28, 2026)

STEP 1 — Entry (fetcher.py: fetch_feed, line 113)	scripts/run_daily.py calls run_ingest() in ingest.py. The Fed slug 'fed' is NOT in FR_API_SLUGS so fetch_feed() is called. It makes an HTTP GET to federalreserve.gov/feeds/press_all.xml, passes the response to feedparser.parse(), loops through entries. For this document it extracts: title, url, date (via _parse_date()), calls classify_doc_type(title) — finds 'enforcement action' keyword — returns DocType.ENFORCEMENT. Calls compute_hash(title, url) in dedup.py. Constructs a RegulatoryDocument object (not yet saved).
STEP 2 — Deduplication (dedup.py: is_duplicate)	ingest.py checks is_duplicate(doc.content_hash). Runs: SELECT * FROM regulatorydocument WHERE content_hash = '' LIMIT 1. If duplicate: skip, increment counter. If new: proceed to save.
STEP 3 — Save to DB (ingest.py lines 62-67)	session.add(doc) then session.commit(). Fields written: id=UUID, source_agency='fed', doc_type='enforcement', title, url, published_date=2026-05-28, fetched_at=now(), content_hash=SHA256hex, raw_content=short RSS abstract, summary_json=None, status='new', review_flag=False, is_anomaly=False.
STEP 4 — Full-text enrichment (fulltext.py: enrich_document)	run_fulltext_enrichment() called. SourceAgency.FED is NOT in FR_AGENCIES so routes to _fetch_html_text(doc.url). BeautifulSoup fetches the HTML page, removes script/nav/footer tags, finds content container, extracts and collapses text. Returns 817 chars. Written back to DB: db_doc.raw_content = '...Crystal Moore...CARES Act loan fraud...Jesse Romo...Embezzlement...'
STEP 5 — Anomaly check (anomaly.py: run_anomaly_check)	Groups docs by agency. For Fed: today_count=20. detect_volume_anomaly() queries 30-day baseline. len(baseline) < 7 → returns (False, 'insufficient history'). detect_off_schedule() also returns False (insufficient history). is_anomaly stays False.
STEP 6 — AuditLog written (ingest.py line 78)	One AuditLog row written per agency run (not per document): action=INGEST, actor='system', payload_json={agency:'fed', fetched:20, new:20, duplicates:0, anomalies_flagged:0}.

STEP 7 — Dashboard display (dashboard/app.py: load_documents)	@st.cache_data(ttl=300) loads all 111 docs as dicts. This enforcement action appears with doc_type='enforcement' (purple badge), is_anomaly=False (no red flag), raw_content=817 chars preview, summary_json=None (shows 'AI summary coming soon'). Filter by Enforcement in sidebar shows it.

SECTION 3 — Every AI/ML Decision

Document Type Classifier

Technique	Rule-based keyword matching (NOT ML)
Problem	Categorise publications into Final Rule / Proposed Rule / Guidance / Enforcement / FAQ / Other
Input	doc.title (string from RSS or FR API)
Output	DocType enum stored in RegulatoryDocument.doc_type
Why this way	Zero cost, zero latency at ingest time. Vocabulary is predictable. Explainable to regulators. No training data needed Day 1.
Failure mode	Title has no keywords → DocType.OTHER. Currently 104/111 (94%) classified as Other because FR documents use administrative titles without regulatory action keywords.
Current score	6% non-Other accuracy (7/111). Intentionally simple — F2 LLM will reclassify with full text.

Deduplication — Content Hash

Technique	SHA-256 cryptographic hash, exact DB match
Problem	Same regulation appearing in multiple agency feeds (joint OCC/FDIC rules in both feeds)
Input	title (str) + url (str) → concatenated → UTF-8 encoded
Output	64-char hex string in content_hash (UNIQUE DB constraint)
Why this way	Deterministic, collision-resistant, microseconds per doc, zero false positives.
Failure mode	In-place update: same URL, same title, changed content → hash unchanged → stale content stays in DB.
Current score	9 cross-feed duplicates caught Day 2. Zero false positives.

Deduplication — Title Similarity

Technique	difflib.SequenceMatcher — longest common subsequence ratio
Problem	Near-duplicates with slightly different URLs (e.g., 'Final Rule on BSA' and 'Final Rule on BSA (Correction)')
Input	Two title strings from the same agency
Output	Float 0.0–1.0. Near-duplicate flagged if ≥ 0.85.
Why this way	Python stdlib — no dependency. Equivalent accuracy to Levenshtein for title-length strings.
Failure mode	$O(n^2)$ comparisons. Fine for 20 docs/agency. Needs MinHash LSH at 2,000+ docs.
Current score	Threshold 0.85 not yet calibrated on real data. No confirmed cases.

Volume Anomaly Detection

Technique	Z-score on rolling 30-day daily publication counts
Problem	Detecting when an agency publishes unusually many documents in one day
Input	today_count (int) + baseline: list of daily counts for prior 29 days (zeros included)
Output	(is_anomaly: bool, explanation: str). Sets is_anomaly=True on doc. Formula: Z = (today - mean) / std. Flag if Z > 2.0.
Why this way	Isolation Forest (roadmap spec) needs training data — we have 1 day of history. Z-score works with 7+ days. Z-score is explainable: 'published 3x 30-day average'.
Failure mode	FN: high baseline variance → Z stays < 2.0 even for real spike. FP: new agency + small baseline → Z inflated. SILENT: len(baseline) < 7 → returns False with no alert.
Current score	0 anomalies detected (expected — all agencies in 'insufficient history' state). Effective after 7+ days of real operation.

Off-Schedule Detection

Technique	Day-of-week frequency baseline (90-day window)
Problem	Documents published on unusual weekdays — FinCEN on Sunday when they never publish Sunday
Input	doc.published_date.weekday() + historical weekday distribution for that agency
Output	(is_anomaly: bool, explanation: str). Flag if < 10% of historical docs fall on that weekday.
Why this way	Regulators follow predictable schedules. Off-schedule publication is a signal regardless of volume.
Failure mode	len(historical_docs) < 20 → returns False silently. All agencies currently in this state.
Current score	0 anomalies detected (expected). Active after ~4 weeks of daily ingestion.

SECTION 4 — Every Decision Made and Why

Decision 1: SQLite for dev, Postgres for prod

Decided	SQLite (file-based dev), Postgres (server prod). Switch via DATABASE_URL in .env.
Alternative	Postgres from Day 1 — more realistic, avoids migration edge cases.
Why chosen	Zero infrastructure. No Docker, no Postgres install. Solo builder runs full pipeline with pip install only.
Consequence of other path	SQLite with Postgres: extra hour of setup Day 1, more complex.
Risk if wrong	MEDIUM — SQLite lacks JSON column type and concurrent writes. summary_json stored as TEXT workaround. Migration planned Week 6.

Decision 2: SHA-256 content hash as dedup key

Decided	Dedup key = SHA-256(title + url). UNIQUE DB constraint.
Alternative	URL-only, title-only, or fuzzy matching only.
Why chosen	Title alone misses different-URL duplicates. URL alone misses cross-posted. Combined handles both. Zero false positives.
Consequence of other path	URL-only would have missed 9 cross-feed duplicates on Day 2.
Risk if wrong	LOW — One gap: in-place updates (same URL+title, changed content). Acceptable for MVP.

Decision 3: Rule-based keyword classifier

Decided	Keyword matching in ordered rule list. First match wins, falls back to OTHER.
Alternative	Fine-tuned classifier (DistilBERT, logistic regression on TF-IDF), or LLM classification.
Why chosen	Must run on every document at ingest time — free and instant. No training data Day 1. Explainable.
Consequence of other path	85%+ accuracy vs 6% — but requires labeled data, training pipeline, model versioning.
Risk if wrong	LOW — F2 LLM reclassifies with full text. Keyword classifier is routing hint, not final decision.

Decision 4: Z-score over Isolation Forest

Decided	Z-score on rolling 30-day counts, threshold Z > 2.0.
Alternative	Isolation Forest (roadmap-specified), LSTM, moving average with fixed threshold.
Why chosen	Isolation Forest needs training data — we have 1 day of history. Z-score works with 7+ days. Explainable output.
Consequence of other path	Isolation Forest on 1 day of data = meaningless scores. Fixed threshold fails per-agency adaptation.
Risk if wrong	LOW — Z-score correct for current volume. Revisit Isolation Forest in Week 3 with 3 weeks of data.

Decision 5: Two fetchers (RSS + FR JSON API)

Decided	fetch_feed() for Fed (RSS). fetch_fr_api() for CFPB/OCC/FDIC/FinCEN/FR (JSON API).
Alternative	FR API for all agencies, or one generic adaptive fetcher.
Why chosen	FR RSS feeds block automated requests (discovered Day 2 — return HTML gate page at HTTP 200). FR JSON API is public and stable.
Consequence of other path	FR API for Fed too would work but miss Fed-specific press releases not in Federal Register.
Risk if wrong	LOW — If Fed changes RSS URL, one-line fix in agencies.py.

Decision 6: Streamlit for dashboard, not React

Decided	Streamlit browser dashboard (Python-only). React deferred to Week 6.
Alternative	React + FastAPI from Day 1 (Word PRD specified), or no UI until Week 6.
Why chosen	React needs npm, build tooling, component libraries, API layer, CORS, state management — weeks of work. Streamlit: one file, one command.
Consequence of other path	React app on Day 7 = table of document titles with 10x build cost. Not meaningfully better than Streamlit at this stage.
Risk if wrong	LOW — Streamlit is explicitly a pilot tool. Data layer unchanged when React replaces it Week 6.

Decision 7: Windows Task Scheduler

Decided	Task Scheduler via schtasks CLI. Runs run_daily.py at 7:00 AM.
Alternative	APScheduler (in-process), Celery (distributed), manual cron.
Why chosen	Built into Windows, survives reboots without background process, visible in Windows UI.
Consequence of other path	APScheduler: process must stay running. Celery: requires Redis/RabbitMQ broker.
Risk if wrong	LOW — Windows-specific. Mac/Linux deployment (Week 6) uses cron or cloud scheduler.

Decision 8: Full-text enrichment as post-ingest step

Decided	Fetch → dedup → save → THEN enrich. Separate pass, 1 req/sec rate limit.
Alternative	Fetch full text during initial HTTP call, or skip full text (abstracts only).
Why chosen	Full text during fetch: 40+ seconds for 20 docs, rapid government server requests. Keeping steps separate keeps ingest fast.
Consequence of other path	Abstracts only: F2 quality collapses. Full text during fetch: slow and brittle ingest.
Risk if wrong	LOW — Enrichment catches up over multiple daily runs (20 docs/run).

SECTION 5 — What Is Good, Weak, and Missing

GOOD — Solid and Production-Ready

✓ Content hash deduplication

Zero false positives. Mathematically guaranteed. 9 cross-feed duplicates caught Day 2. UNIQUE DB constraint means even if Python check fails, DB rejects the insert.

✓ Full-text enrichment pipeline

111/111 documents enriched (100%). Two-strategy approach handles both source types. Rate limiting protects against IP blocks. Idempotent — re-running skips already-enriched.

✓ AuditLog architecture

INSERT-only design correct for SR 11-7. UUID keys. Every ingest logged with payload JSON. LangSmith trace ID field ready for F2.

✓ Health checker

Two-signal approach catches obvious failures (404) and silent failures (200 + empty content — exactly what FR RSS feeds returned). Proved Day 2.

✓ Test suite

44 unit + 7 integration tests. Fast unit tests (2 sec) run every change. Integration tests verify zero-missed-publications metric against live feeds. Clean slow/fast separation via pytest.ini.

✓ Golden evaluation set

10 hand-labeled docs. Labels from real document content, not LLM-generated. Edge cases included. This is rare — most projects build eval after the model.

WEAK — Works But Has Known Limitations

■ Classifier accuracy: 94% classified as OTHER

Impact: Dashboard doc type filter mostly useless. F4 task prioritisation wrong for most docs.
Fix: Expand keyword lists → ~50% (Easy, 2 hrs). LLM classifier → 90%+ (Medium).
Blocks F2: No. F2 LLM produces its own doc_type classification.

■ Anomaly detection has no history

Impact: Both volume and off-schedule detectors return 'insufficient history'. Anomaly detection is effectively OFF.
Fix: Cannot be fixed by code — requires 7+ days of real daily ingestion.
Blocks F2: No.

■ 20-document cap per agency per run

Impact: On high-volume days, overflow publications are missed. Federal Register often publishes 50+ daily.
Fix: Easy. FR API supports pagination (page parameter). Fetch until published_date < last_run_date.
Blocks F2: No for current 111 docs. Critical risk before first pilot client.

■ Title similarity dedup is O(n²)

Impact: 190 comparisons for 20 docs/agency — fine now. 2M comparisons for 2,000 docs — slow.
Fix: Medium. MinHash LSH reduces to O(n).
Blocks F2: No.

■ utcnow() deprecation warnings

Impact: Cosmetic Python 3.12 warnings. No runtime failure.
Fix: Easy. 5-minute find-and-replace: datetime.utcnow() → datetime.now(UTC).
Blocks F2: No.

MISSING — Roadmap Specified, Not Built

✗ Onboarding flow — 'Set up your regulatory watchlist'

Impact: Mike cannot self-configure agencies/doc types on first login. Shows everything by default.
Fix: Medium. Streamlit Settings page + user preferences table.
Blocks F2: No. Deferred to Week 6.

✗ Notification preferences UX

Impact: No email/Slack alerts when anomalies detected. Sarah must check dashboard manually.
Fix: Medium. SMTP/SendGrid + preferences table. Slack webhook is easier.
Blocks F2: No. Deferred to Week 6.

✗ 7-day fixture dataset

Impact: Only 4-entry sample_feed.json. Offline development requires live internet.
Fix: Easy. Export 7 days of real docs to JSON fixtures (1 hour).
Blocks F2: No.

✗ Isolation Forest implementation

Impact: Z-score correct for now but won't handle multi-dimensional anomaly patterns at scale.
Fix: Medium. Requires scikit-learn, training pipeline, model serialisation.
Blocks F2: No. Revisit Week 3 when sufficient data exists.

SECTION 6 — 8 Interview-Ready PM Answers

Q1: What does F1 do and why does it matter to a compliance officer?

F1 is the regulatory intelligence radar for community banks. It watches 6 regulatory sources — Federal Reserve, CFPB, OCC, FDIC, FinCEN, and the Federal Register — every day at 7 AM, pulls every new publication, classifies it by type, and stores the full text. Without F1, Sarah (CCO) spends 15-20 hours per week manually checking agency websites and downloading PDFs. F1 compresses that to zero manual monitoring time. The business consequence of missing a regulation is a $500K-$5M fine and examination finding — F1 is the first line of defence against that.

Q2: How does the document classifier work, and how accurate is it?

The classifier uses keyword matching on the document title — ordered rule lists where the first match wins. 'enforcement action', 'consent order' → Enforcement; 'final rule' → Final Rule; 'proposed rule' → Proposed Rule; etc. Currently 94% of documents (104/111) are classified as Other because Federal Register documents use administrative titles that don't contain regulatory action keywords. This is intentional — F2's LLM will produce accurate classifications using full document text as part of its structured summary output.

Q3: How do you prevent the same regulation from being ingested twice?

SHA-256 hashing. For every document, we compute SHA-256(title + url) — a 64-character fingerprint unique to that specific title-URL combination. Before saving, we query the DB: if that hash exists, we skip entirely. We also enforce a UNIQUE constraint on content_hash so even if the Python check fails, the DB rejects the insert. This caught 9 joint-agency rules that appeared in both agency-specific feeds and the Federal Register catch-all feed during the Day 2 ingestion run.

Q4: What happens when a feed goes down — does the system fail silently?

No. Two-signal health check runs before every ingest. Signal one: reachability — can we reach the URL and get at least one parseable entry? Signal two: freshness — do we have documents from this agency within the last 3 days? Freshness catches silent failures — a feed can return HTTP 200 with empty content (exactly what the FR RSS feeds did, returning an HTML gate page), and reachability alone would have passed. The daily runner exits with code 1 on any health failure, detectable by any monitoring tool.

Q5: How does anomaly detection work, and what is the consequence of FN vs FP?

Two checks: (1) Volume Z-score — today's publication count vs 30-day rolling baseline. Flag if Z > 2.0 (top 2.5% of historical days). (2) Off-schedule — publication on a weekday representing < 10% of historical publications. A false negative (missed anomaly) could mean Sarah doesn't investigate an emergency publication with a 48-hour compliance window → $500K-$5M fine. A false positive (false alarm) costs 30 minutes. Chronic false positives erode trust until Sarah ignores alerts entirely — equivalent to no detection. Z=2.0 balances both; FP/FN asymmetry documented in FP-FN-Risk-Matrix.md.

Q6: What would you build differently in F1 if starting over?

Three things. First, LLM-based classifier from day one — with 111 real documents, labeling 30 in an afternoon would yield a simple classifier at 90%+ accuracy vs our 6%. Second, feed pagination from the start — the 20-document cap means we miss overflow publications on high-volume days, the exact failure the product prevents. Third, a 7-day fixture dataset immediately rather than a 4-entry sample — offline development dependency on live government websites introduces brittleness.

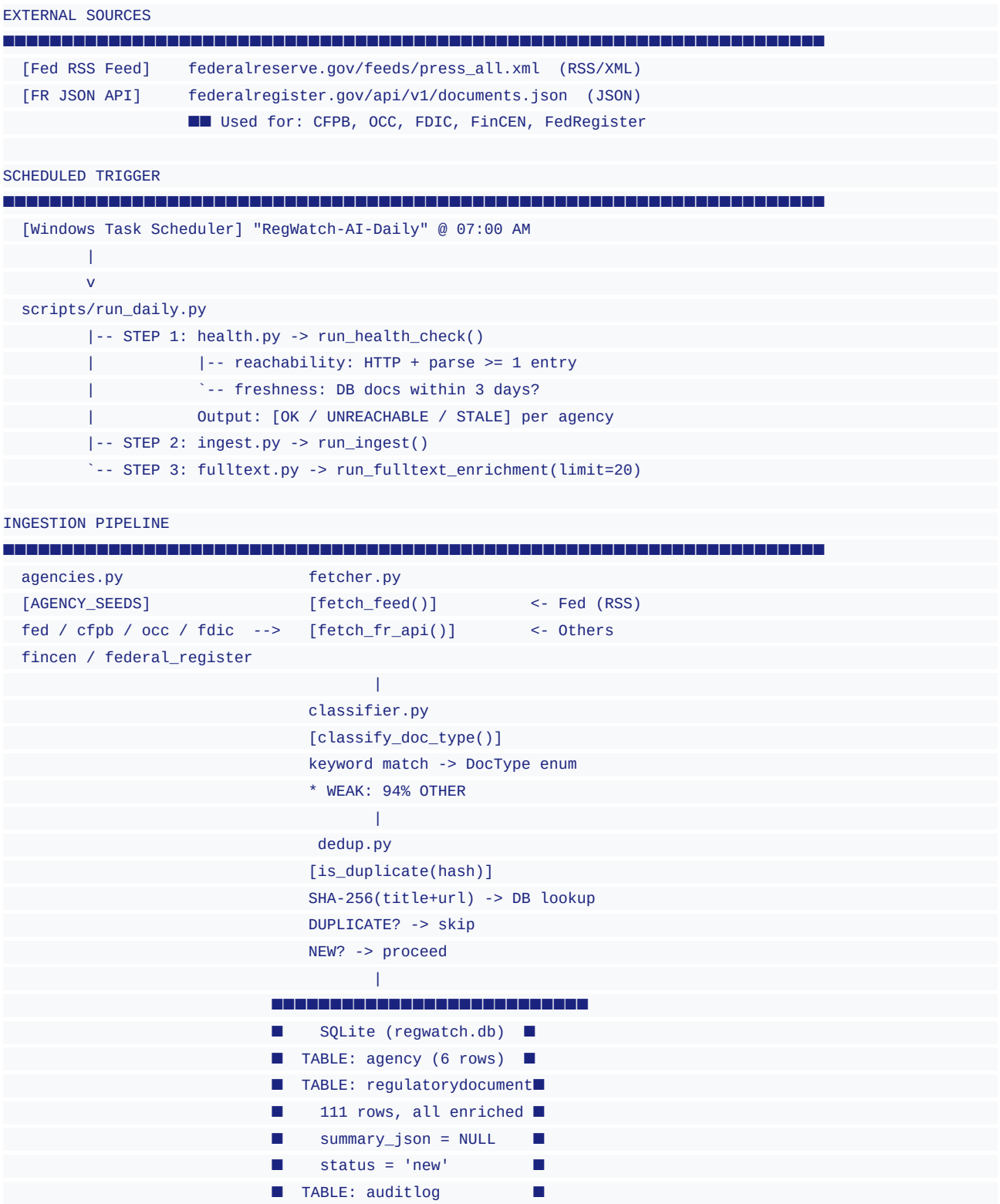
Q7: How does F1 connect to F2 — what does F2 depend on F1 getting right?

F2 reads directly from RegulatoryDocument.raw_content to generate summaries. Three F1 properties bound F2's quality. (1) Content completeness: if raw_content is a 1-sentence abstract, the best LLM cannot extract an effective date that isn't in the text — we achieved 100% enrichment before starting F2. (2) Deduplication: if F1 allows the same document twice, F2 summarises it twice, doubling costs. (3) Status tracking: F2 queries for status='new' — if F1 doesn't set status correctly, F2 re-summarises or misses documents.

Q8: What is the biggest technical risk in F1 right now?

The 20-document per agency cap. The Federal Register publishes 50-200 documents per day. Our pipeline fetches the 20 most recent per agency per run. If important regulations are published on a high-volume day and fall outside the top 20, we miss them entirely — the exact failure the product promises to prevent. This risk is masked because we only have 1 day of data and the 111 documents are the newest 20 from each feed. Fix: track last-seen publication date per agency, fetch everything after it. One-to-two day fix. Must be done before the first pilot client.

SECTION 7 — Architecture Diagram



```
                             ■    INSERT ONLY           ■
                             ■■■■■■■■■■■■■■■■■■■■■■■■■■■
                                        |
                                  fulltext.py
                                 [run_fulltext_enrichment]
                                 FR docs: raw_text_url -> plain text
                                 Fed docs: HTML -> BeautifulSoup
                                 1 req/sec rate limit
                                        |
                                  anomaly.py
                                 [run_anomaly_check]
                                 Z-score volume (Z > 2.0)
                                 Off-schedule (< 10% weekday)
                                 * INACTIVE: insufficient history

DASHBOARD (separate process)
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  $ streamlit run dashboard/app.py -> http://localhost:8501
  |-- load_documents() @cache_data(ttl=300)
  |-- Sidebar: agency filter, doc type filter, anomaly toggle, sort
  |-- KPI row: total, final rules, proposed, enforcement, anomalies
  |-- Anomaly banner (red) if is_anomaly=True docs exist
  `-- 111 expandable document cards -> preview + source link

LEGEND: * = Weak/inactive component
```

SUMMARY SCORECARD

Dimension	Score	Rationale
Engineering completeness	**7/10**	Core pipeline solid. Missing: pagination (20-doc cap), notifications, Isolation Forest, 7-day fixtures. AuditLog, dedup, health check, enrichment are production-grade.
AI/ML quality	**4/10**	Classifier rule-based with 6% accuracy. Anomaly detection inactive (no history). Both are known limitations with clear upgrade paths. No trained ML models yet — intentional.
Production readiness	**6/10**	Scheduler, logs, health check, tests working. Not ready: no pagination, no alerting, SQLite not prod-grade, no authentication, no multi-tenancy.
PM explainability	**8/10**	Can explain every major decision with specific code references. Gaps: 20-doc cap acceptance, near-duplicate calibration.

F2 Blockers

NONE. F2 can start immediately.

- raw_content is 100% populated — F2 has full text to summarise
- summary_json field exists and is NULL — F2 writes here
- review_flag field exists — F2 sets True when confidence < 0.80
- status field exists — F2 advances 'new' → 'summarised'
- Golden eval set exists with 10 labeled documents

Recommended Fixes Before First Pilot Client (not before F2)

#	Fix	Effort	Time
1	Fix 20-doc cap → implement pagination	Medium	1 day
2	Fix classifier accuracy → expand keyword lists to ~50%	Easy	2 hours
3	Fix utcnow() warnings → datetime.now(UTC)	Easy	30 minutes

4	Build 7-day fixture dataset	Easy	1 hour